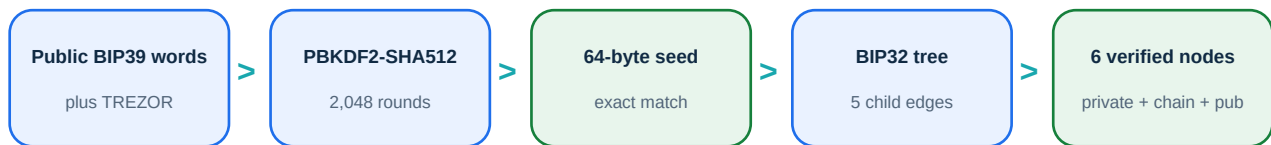


SARA 3.6.3 CUDA Conformance

An ELI5 report on what genuinely worked, measured performance, feasibility, and the exact remaining engineering and misuse-boundary gaps.

VERDICT: THE FIXED-VECTOR CRYPTOGRAPHIC PIPELINE WORKS



Question	Evidence-based answer
Did real CUDA execute?	Yes. Native sm_120 code ran on the NVIDIA GeForce RTX 5070 Ti Laptop GPU.
Were the outputs correct?	Yes. BIP39 and all six BIP32 vector-1 nodes matched published bytes on CPU and GPU.
Was it repeatable?	Yes. Every output in seven large timed runs matched byte-for-byte.
Was it actually accelerated?	Yes for large parallel batches: 76.6x BIP39 and 547.8x BIP32 throughput versus this single CPU implementation.
Is this a wallet recovery tool?	No. It has no candidate source, arbitrary secret input, address target, search loop, or target comparison.

Measured on 2026-08-18 in Europe/Amsterdam. Report inputs are the saved benchmark JSON files, native run logs, compiler resource report, CTest result, and the unchanged SARA 3.6.3 Python implementation.

1. ELI5: what was tested?

Imagine a very precise recipe. The same ingredients must always make the same cake. BIP39 turns a public example phrase and passphrase into a 64-byte seed. BIP32 then grows a family tree of keys from that seed. Every branch has an exact expected answer.

The test gave both the CPU and GPU the same published recipe. It checked the answer after BIP39, then checked the private scalar, chain code, and compressed public key at every BIP32 node. A one-byte difference anywhere would fail the run.

Why repeat one public example thousands of times?

A GPU wins by doing many independent jobs at once. Repeating the same immutable public vector measures parallel throughput without adding a candidate list or a target-search mechanism. It is like timing 65,536 identical, openly published math worksheets - not guessing a private worksheet.

What 'literal SARA' means here

The CUDA functions follow the same transitions as `mnemonic_to_seed`, `master_node_from_seed`, `ckd_private`, and `compressed_public_key` in `src/ugts36/sara363.py`. A separate script evaluated the same fixtures through that unchanged Python source, and every node passed.

This benchmark is not an SGP4 orbital simulation and makes no satellite-navigation claim. The earlier CUDA project contributed its proven Windows/CUDA build and timing patterns only.

2. Exact correctness evidence

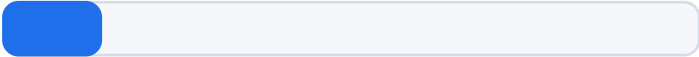
Gate	Coverage	Result
BIP39 CPU	PBKDF2-HMAC-SHA512, 2,048 rounds, 64-byte published seed	PASS
BIP39 GPU	All 65,536 outputs in each of 7 large runs	PASS
BIP32 CPU	Master plus 5 edges; hardened and normal; 6 public vector nodes	PASS
BIP32 GPU	All 6 nodes, then all 4,096 complete paths in each of 7 runs	PASS
Repeatability	Byte-for-byte outputs across timed runs	PASS
CTest	Native quick conformance test through CMake/CTest	1/1 PASS
SARA pytest	Targeted upstream tests/test_sara363.py	42/42 PASS
Literal SARA	Independent fixed-vector execution through unmodified sara363.py	PASS

The important distinction: a program can run without being correct. Here, correctness is tied to external published bytes, not merely to a zero exit code or CPU/GPU agreement.

3. Measured performance

Hardware: **NVIDIA GeForce RTX 5070 Ti Laptop GPU**, compute capability 12.0, CUDA driver API 13010, CUDA runtime 12080. Build: Visual Studio 2022, CUDA 12.8, native and virtual sm_120 code, Release configuration.

Workload	CPU mean	1 GPU item	Large GPU batch	Large throughput
BIP39 seed	1.995 ms/op	61.425 ms	1705.628 ms / 65,536	38,423.4 ops/s
BIP32 vector-1 path	3.513 ms/path	26.174 ms	26.268 ms / 4,096	155,933.3 paths/s

BIP39 seed derivation  **76.6x**

BIP32 full vector path  **547.8x**

Large-batch throughput relative to this single CPU implementation

BIP39 also sustained **78,691,102.2 PBKDF2 rounds/s**. The BIP32 batch sustained **779,666.7 child-derivation edges/s** across the five-edge path.

ELI5 interpretation: one child doing one worksheet on the GPU is slow because the whole classroom must open first. Give the classroom thousands of worksheets, and many children work at the same time. That is why single-item GPU latency loses to the CPU while large-batch throughput wins strongly.

Observed scaling

Scale point	BIP39 ops/s	BIP32 paths/s
1 item	16.3	38.2
Medium batch	36,104.6 (8,192 items)	19,315.6 (512 items)
Large batch	38,423.4 (65,536 items)	155,933.3 (4,096 items)

4. What seriously works - and what that proves

Proven	Not proven by this test
CUDA can reproduce the published BIP39 seed exactly.	Arbitrary-language Unicode normalization on the GPU.
CUDA can generate BIP32 master and private child nodes.	Full wallet-format compatibility or device-wallet import/export.
Normal BIP32 edges work because secp256k1 compressed public keys are produced on-device.	BIP44, BIP49, BIP84, legacy address projection, or blockchain discovery.
Both hardened and normal edges match the complete official vector-1 path.	All official vectors, malformed-input behavior, or invalid-child retry coverage.
Large fixed batches get real GPU throughput acceleration.	Candidate generation, target matching, wallet recovery, or useful search coverage.

So: yes, the implemented cryptographic route seriously works for the published inputs and operations tested. No, that result must not be inflated into a claim that an arbitrary wallet, address type, recovery scenario, or search space is solved.

Compression claim

This benchmark tests deterministic cryptographic transformations, not data compression. It does not measure compression ratio, decompression fidelity, geometric reconstruction, or visual output. Any earlier compression result remains a separate claim and is neither strengthened nor weakened by this BIP39/BIP32 run.

5. Exact remaining engineering gaps

Gap	Why it matters	Current status
General NFKD	BIP39 requires Unicode NFKD for arbitrary text.	Frozen ASCII fixture is already normalized.
Mnemonic validation	Word-list membership and checksum matter before seed derivation.	Handled by SARA Python, not this GPU kernel.
More BIP vectors	Leading-zero and invalid-edge cases catch subtle arithmetic bugs.	Only BIP32 vector 1 and one BIP39 vector are pinned.
Extended-key serialization	xprv/xpub include versions, depth, fingerprints, child numbers, and Base58Check.	Raw node fields are checked; serialization is absent.
Wallet/address layers	Real wallet compatibility needs path conventions, HASH160, scripts, and address encodings.	Deliberately absent.
Production optimization	Resource pressure limits occupancy and wastes throughput.	Correctness-first baseline only.
Security review	Secret-handling code needs constant-time review, zeroization, fuzzing, and independent audit.	Not performed; do not use with secrets.
Search/recovery layer	Would require candidates, arbitrary inputs, projections, and target comparison.	Intentionally not implemented.

6. GPU engineering diagnosis

The compiler's own resource report explains why this is a baseline, not an optimized library:

Kernel	Registers/thread	Stack/thread	Spills
BIP39 batch	254	4,304 bytes	128-byte stores + 128-byte loads
BIP32 batch	253	4,080 bytes	0 reported
BIP32 conformance	232	4,080 bytes	0 reported

Most likely optimization order

1. Reduce SHA-512 message-schedule and HMAC temporary lifetime so fewer registers and local bytes remain live.
2. Precompute immutable HMAC ipad/opad states for the public benchmark fixture.
3. Replace the simple field reducer and generic inversion with reviewed secp256k1-specific routines.
4. Tune block size from measured occupancy and profile memory stalls, instruction mix, and achieved occupancy.
5. Separate conformance code from optimized throughput kernels while retaining the same external-vector gates.
6. Add all official BIP32 edge-case vectors plus differential and randomized tests before trusting any optimization.

These changes could improve the fixed-vector primitive benchmark. They do not require or imply adding a credential-search interface.

7. How far the misuse boundary moved

Present in delivered code	Still absent
GPU SHA-512 and HMAC-SHA512	Runtime mnemonic/passphrase input
GPU PBKDF2-HMAC-SHA512/2048	Candidate lists, masks, mutation, or enumeration
GPU BIP32 master and private CKD	User-selectable derivation paths
GPU secp256k1 compressed public projection	Address projection and arbitrary target input
Parallel repetition of one public vector	Target comparison, discovery loop, or recovery output
Measured primitive throughput	Blockchain access, signing, or transactions

This is the exact boundary for this delivery. It shows that deterministic wallet primitives can be GPU-native and fast in parallel. It deliberately stops before the components that would turn primitive evaluation into credential search. This describes what this assistant delivered; it is not a universal claim about every AI system's policies.

8. Reproduction and evidence map

One-command Windows reproduction

```
& .\scripts\build_windows.ps1
```

The script pins Visual Studio 2022, CUDA 12.8, sm_120, Release mode, CTest, the full measured benchmark, and the independent literal-SARA check. Its external build directory is intentionally short to avoid the Windows/MSBuild path-length failure observed during validation.

Artifact	Purpose
include/sara363_cuda/crypto.cuh	Portable host/device SHA-512, HMAC, PBKDF2, secp256k1, and BIP32 operators
src/main.cu	Immutable fixtures, CUDA kernels, exact checks, timing, repeatability, and JSON output
scripts/verify_against_sara.py	Independent execution through the unchanged literal SARA 3.6.3 source
output/benchmark_metrics_max_batch.json	Large-batch machine-readable metrics used in this report
output/benchmark_metrics_single_item.json	Single-item latency evidence
output/max_batch_benchmark_run.txt	Human-readable full run and PASS evidence
docs/TRACEABILITY.md	Literal SARA operator mapping
docs/SAFETY_SCOPE.md	Structural input/search boundary audit

9. Final answer in plain language

It works: the GPU computed the intended BIP39 and complete BIP32 vector-1 transformations correctly, on real hardware, repeatedly, and substantially faster in large batches than this single CPU implementation.

It is not magic: a single job is much slower on this GPU baseline, the kernels are resource-heavy, and correctness has only been established for the pinned public vectors and covered operators.

It is not wallet recovery: the delivered program cannot accept a private phrase, generate guesses, choose arbitrary paths, derive wallet addresses, accept a target, or search for a match. The measured rate is cryptographic primitive throughput, not a recovered-wallet claim.

The honest feasibility conclusion is therefore: GPU-native SARA seed and HD-tree primitives are feasible and already functionally correct as a conformance baseline. Production-grade wallet compatibility, security hardening, broader conformance, and performance tuning remain real work. Credential-search functionality is outside this delivery.

Sources

Bitcoin BIP 39: <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>

Trezor BIP39 vectors: <https://github.com/trezor/python-mnemonic/blob/master/vectors.json>

Bitcoin BIP 32 test vectors: <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki#test-vectors>

Local literal source: src/ugts36/sara363.py in the SARA 3.6.3 package

Local measured evidence: cuda_conformance/output/*.json and *.txt

Measurement caveat: timings are observations from one laptop state, compiler build, batch configuration, and power/thermal condition. They are not guaranteed performance on other machines. No statistical confidence interval or profiler-derived occupancy measurement is claimed.